



Język Python
Wykład 01

Fizyka Medyczna

06 października 2026

Wprowadzenie do Pythona

Jacek Gatlik

e-mail: jacek.gatlik@agh.edu.pl

www: <https://galaxy.agh.edu.pl/~jgatlik/>

Wydział Fizyki i Informatyki Stosowanej,
Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

Plan wykładu

Informacje wstępne

Plan wykładów i materiały dodatkowe

Historia i znaczenie języka Python

Uruchamianie Pythona

Podstawowe typy danych, operacje wejścia wyjścia

Operatory

Wcięcia i instrukcje warunkowe

Przykłady

Pętle

Przykłady

Zadania dodatkowe

Sekcja 1

Informacje wstępne

Warunki zaliczenia

- ▶ Każde laboratorium, z wyjątkiem pierwszego, rozpoczyna się krótką kartkówką z materiału omawianego na ostatnim wykładzie (chyba, że zapowiedziano inaczej).
- ▶ Za kartkówkę można uzyskać **25% punktów** przewidzianych za dane zajęcia, a pozostałe **75% punktów** przyznawane jest za wykonanie i zaprezentowanie programu przygotowanego w czasie zajęć. Na pierwszych zajęciach laboratoryjnych wszystkie punkty przyznawane są za wykonanie zadania praktycznego.
- ▶ Maksymalna liczba punktów przewidziana za poszczególne zajęcia lub zadania może być zróżnicowana w zależności od ich zakresu i nakładu pracy oraz jest podawana studentom przed ich realizacją.
- ▶ Podstawowym terminem uzyskania zaliczenia jest koniec zajęć w danym semestrze. W przypadku braku zaliczenia student może poprawić wyniki z dwóch laboratoriów.
- ▶ Ocena końcowa jest tożsama z oceną uzyskaną z ćwiczeń laboratoryjnych. Jest ona ustalana na podstawie sumy punktów uzyskanych przez studenta w trakcie semestru w stosunku do maksymalnej liczby punktów możliwych do zdobycia. Uzyskany wynik procentowy jest przeliczany na ocenę zgodnie ze skalą określoną w Regulaminie Studiów AGH (§ 13 ust. 1).

Nieobecności i zadania dodatkowe

- ▶ Usprawiedliwiona nieobecność wymaga samodzielnego opanowania materiału oraz zaliczenia go w formie i terminie uzgodnionych z prowadzącym. W przeciwnym razie student otrzymuje **0 punktów za dane laboratorium**.
- ▶ Nieobecność nieusprawiedliwiona oznacza uzyskanie **0 punktów za dane laboratorium**, bez możliwości jego poprawy.
- ▶ Student, który bez usprawiedliwienia opuścił więcej niż 20% zajęć, a jego cząstkowe wyniki w nauce są negatywne, może zostać pozbawiony przez prowadzącego możliwości poprawkowego zaliczania zajęć.
- ▶ Na wybranych wykładach będą pojawiały się zadania dla chętnych, za które można uzyskać dodatkowe **1–3 punkty**.
- ▶ Punkty dodatkowe będą doliczane do wyniku najbliższego laboratorium.
- ▶ Rozwiązanie w postaci kodu należy przelać na mój adres e-mail do godziny **23:59 w niedzielę następującą po wykładzie**.
- ▶ Zadania są dobrowolne, a ich niewykonanie **nie wpływa na ocenę końcową**.

Samodzielność rozwiązań i narzędzia AI

Student odpowiada za każde oddane rozwiązanie, niezależnie od tego, z jakich materiałów lub narzędzi pomocniczych korzystał. Zakres narzędzi i materiałów pomocniczych dozwolonych podczas rozwiązywania zadań zostanie szczegółowo omówiony na ćwiczeniach.

W przypadku wątpliwości dotyczących samodzielności lub zrozumienia rozwiązania student może zostać poproszony o:

- ▶ wyjaśnienie działania kodu i uzasadnienie wybranych rozwiązań,
- ▶ wskazanie roli wybranych instrukcji lub zmiennych,
- ▶ wprowadzenie niewielkiej modyfikacji do programu.

Brak umiejętności wyjaśnienia własnego rozwiązania skutkuje przyznaniem **0 punktów za dane laboratorium**; w przypadku zadania dodatkowego – przyznaniem 0 punktów za laboratorium, do którego miały zostać doliczone punkty dodatkowe. Wynik ten nie podlega poprawie.

W czasie zajęć laboratoryjnych nie można korzystać z narzędzi AI.

Sekcja 2

Plan wykładów i materiały dodatkowe

Plan wykładu

1. **Podstawy języka Python:** składnia, typy danych, operatory, instrukcje sterujące i podstawowe operacje wejścia/wyjścia.
2. **Struktury danych i przetwarzanie kolekcji:** napisy, listy, krotki, słowniki, zbiory, iterowanie oraz wyrażenia składane.
3. **Funkcje i organizacja programu:** definiowanie i wywoływanie funkcji, argumenty, wartości zwracane, zakres zmiennych, funkcje anonimowe, iteratory i generatory.
4. **Moduły, pakiety i niezawodność programów:** biblioteka standardowa, tworzenie i wykorzystanie modułów, obsługa wyjątków oraz podstawowe zasady organizacji kodu.
5. **Praca z danymi i plikami:** pliki tekstowe i binarne, formaty CSV i JSON, ścieżki i katalogi oraz wyrażenia regularne.

Plan wykładu

6. **Programowanie obiektowe w Pythonie:** klasy i obiekty, atrybuty i metody, dziedziczenie, enkapsulacja oraz metody specjalne.
7. **Obliczenia naukowe i numeryczne:** wykorzystanie bibliotek NumPy i SciPy do przetwarzania danych, obliczeń macierzowych, analizy numerycznej i optymalizacji.
8. **Analiza i wizualizacja danych:** wykorzystanie bibliotek pandas i Matplotlib oraz wybranych narzędzi do przetwarzania, analizy i prezentacji danych.
9. **Efektywne programowanie w Pythonie:** wektoryzacja obliczeń, pomiar wydajności i profilowanie kodu, wykorzystanie pamięci oraz wybrane techniki przyspieszania obliczeń.
10. **Wprowadzenie do uczenia maszynowego i sieci neuronowych w Pythonie:** przygotowanie danych, podstawowe modele uczenia maszynowego, trenowanie i ocena modeli oraz podstawy budowy sieci neuronowych.

Literatura do kursu

Literatura obowiązkowa

1. Eric Matthes: *Python. Instrukcje dla programisty*, wydanie III. Helion, Gliwice, 2023.
2. Robert Johansson: *Matematyczny Python. Obliczenia naukowe i analiza danych z użyciem NumPy, SciPy i Matplotlib*. Helion, Gliwice, 2021.
3. Jake VanderPlas: *Python Data Science Handbook: Essential Tools for Working with Data*, 2nd ed. O'Reilly Media, 2023.

Literatura dodatkowa

1. Allen B. Downey: *Think Python: How to Think Like a Computer Scientist*, 3rd ed. O'Reilly Media, 2024.
2. Brett Slatkin: *Effective Python: 125 Specific Ways to Write Better Python*, 3rd ed. Addison-Wesley Professional, 2024.
3. Micha Gorelick, Ian Ozsvald: *High Performance Python: Practical Performant Programming for Humans*, 2nd ed. O'Reilly Media, 2020.
4. Aurélien Géron: *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. O'Reilly Media, 2022.

Dodatkowe ciekawe materiały

- ▶ CheckiO – gra oparta na rozwiązywaniu krótkich zadań programistycznych w Pythonie. Zadania mają różne poziomy trudności i pozwalają porównywać swoje rozwiązania z kodem napisanym przez innych użytkowników.
- ▶ CodinGame – platforma łącząca programowanie z grami i zagadkami logicznymi. Kod napisany w Pythonie steruje m.in. postaciami, pojazdami i botami, dzięki czemu efekty programu są od razu widoczne.
- ▶ Reeborg's World – interaktywne środowisko, w którym programujemy zachowanie robota poruszającego się po planszy. Bardzo dobre miejsce do przećwiczenia instrukcji warunkowych, pętli i funkcji.
- ▶ CodingBat – duży zestaw krótkich ćwiczeń z Pythona dotyczących m.in. napisów, list, instrukcji warunkowych i pętli. Dobry sposób na kilka minut regularnego treningu pomiędzy zajęciami.
- ▶ Exercism – Python Track – rozbudowana ścieżka ćwiczeń prowadząca od podstaw języka do bardziej zaawansowanych zagadnień. Rozwiązania są automatycznie testowane, a platforma pozwala również zobaczyć inne sposoby rozwiązania problemu.

Dodatkowe ciekawe materiały

- ▶ Python Tutor – narzędzie wizualizujące wykonywanie programu krok po kroku. Pokazuje zmiany wartości zmiennych, obiekty, wywołania funkcji i strukturę pamięci, co bardzo pomaga w zrozumieniu działania kodu.
- ▶ CodeCombat – gra, w której sterowanie bohaterem odbywa się poprzez pisanie kodu. Kolejne poziomy stopniowo wprowadzają podstawowe konstrukcje programistyczne i pozwalają obserwować działanie programu bezpośrednio w świecie gry.
- ▶ The Python Challenge – seria nietypowych zagadek logicznych, w których rozwiązanie kolejnych poziomów wymaga odkrycia wskazówki i napisania odpowiedniego fragmentu kodu w Pythonie. Bardziej przypomina internetowy escape room niż kurs programowania.
- ▶ Advent of Code – coroczny “kalendarz adwentowy” z fabularnymi zagadkami programistycznymi. Zadania można rozwiązywać w dowolnym języku, a wcześniejsze edycje tworzą ogromne archiwum problemów o bardzo zróżnicowanym poziomie trudności.
- ▶ Project Euler – klasyczny zbiór problemów łączących matematykę z programowaniem. Świetny dla osób, które lubią teorię liczb, ciągi, kombinatorykę i szukanie sprytniejszych rozwiązań zamiast prostego brute force.

Dodatkowe ciekawe materiały

- ▶ Rosalind – programowanie połączone z bioinformatyką. Zamiast abstrakcyjnych przykładów pojawiają się m.in. sekwencje DNA i RNA, mutacje, genetyka i analiza danych biologicznych. Dostępna jest także specjalna seria prostych zadań *Python Village*.
- ▶ Codewars – zadania programistyczne przedstawione jako trening sztuk walki: rozwiązujemy kolejne *kata*, zdobywamy wyższe stopnie i możemy porównywać własny kod z rozwiązaniami innych osób. Dobry sposób na odkrywanie różnych stylów pisania tego samego programu.
- ▶ Kattis – ogromny zbiór problemów używanych m.in. w zawodach programistycznych. Zadania mają określony poziom trudności, a rozwiązanie wysyła się do automatycznego systemu oceniającego. Można zacząć od bardzo prostych problemów i stopniowo przechodzić do prawdziwych wyzwań algorytmicznych.
- ▶ CodeAbbey – zbiór kilkuset krótkich problemów programistycznych, od absolutnych podstaw po znacznie bardziej nietypowe zagadki. Oprócz klasycznych algorytmów pojawiają się zadania związane z matematyką, fizyką, elektroniką czy muzyką.
- ▶ OverTheWire: Bandit – nie jest to kurs Pythona, lecz bardzo ciekawa gra ucząca pracy w terminalu Linux. Każdy poziom wymaga odnalezienia hasła do następnego, wykorzystując po drodze polecenia systemowe, SSH, pliki, kodowanie i podstawowe narzędzia Unix.

Sekcja 3

Historia i znaczenie języka Python

Python – krótka historia

- ▶ Guido van Rossum rozpoczął prace nad Pythonem w CWI w Niderlandach pod koniec 1989 roku.
- ▶ Pierwsza publicznie udostępniona wersja, Python 0.9.0, pojawiła się w lutym 1991 roku.
- ▶ Nazwa języka nawiązuje do brytyjskiego programu komediowego *Monty Python's Flying Circus*, a nie do węża.
- ▶ Python 2.0 został wydany w 2000 roku.
- ▶ Python 3.0 ukazał się w grudniu 2008 roku i celowo nie zachowywał pełnej zgodności wstecznej z linią 2.x.
- ▶ Oficjalne wsparcie dla Pythona 2 zakończyło się 1 stycznia 2020 roku. Na kursie korzystamy wyłącznie z **Pythona 3**.

Źródła: Python FAQ, Python 3.0 Release, Sunsetting Python 2.

Worldwide, Jul 2026 :

Rank	Change	Language	Share	1-year trend
1		Python	47.49 %	+16.5 %
2		Java	11.44 %	-4.2 %
3		C/C++	9.68 %	+1.5 %
4	↑↑	R	4.68 %	-0.4 %
5	↓	JavaScript	3.97 %	-3.4 %
6	↑↑↑↑↑	Objective-C	3.15 %	+0.8 %
7		PHP	2.29 %	-1.2 %
8	↓↓↓	C#	2.06 %	-3.2 %
9	↓	Rust	2.06 %	-0.6 %
10		Swift	1.98 %	-0.4 %

PYPL Popularity of Programming Language Indeks powstaje na podstawie analizy częstotliwości wyszukiwania haseł związanych z kursami w Google. Źródło: <https://pyp1.github.io>

Most commonly used languages in AI-tagged projects



1 Python

Powers nearly half of all new AI repos (**582k; +50.7% YoY**), cementing its role as the backbone of AI work-from training and inference to orchestration and deployment.

2 JavaScript

With **88k repos (+24.8% YoY)**, it remains a major bridge for dashboards, demos, and lightweight integrations around model endpoints.

3 TypeScript

The fastest-growing front-end language by volume (**86k repositories; +77.9% YoY**), reflecting demand for typed SDKs and safer API clients in production apps.

4 Shell

Though smaller in absolute terms (**9k repos**), Shell saw the **largest percentage growth (324%)**, underscoring the rise of scripted evals, data prep, and deployment pipelines.

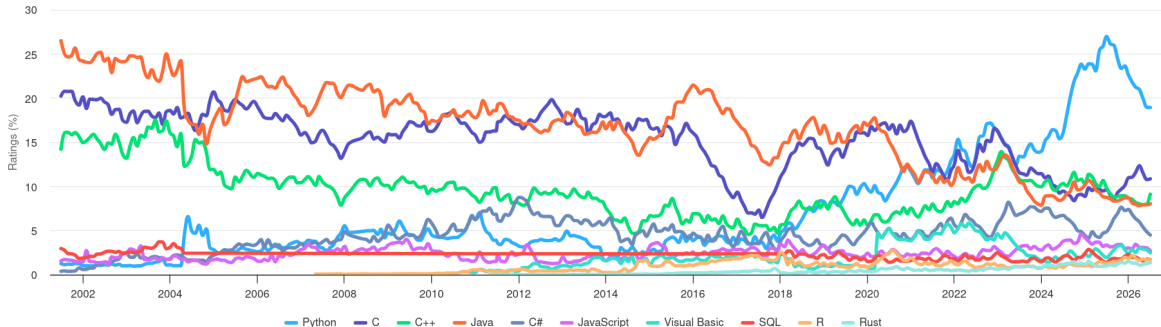
5 C++

Now above **7.8k repos (+11.0% YoY)**, showing its continued importance in performance-critical inference engines, runtimes, and hardware-close systems.

Najczęściej używane języki w zastosowaniach AI. Źródło: <https://github.blog>

TIOBE Programming Community Index

Source: www.tiobe.com



TIOBE Programming Community indeks stanowi wskaźnik popularności języków programowania. Źródło:
<https://www.tiobe.com/>

Dlaczego Python w nauce?

- ▶ Czytelna składnia pozwala skupić się na rozwiązywanym problemie, a nie na technicznych szczegółach języka.
- ▶ Rozbudowany ekosystem obejmuje narzędzia do obliczeń numerycznych, analizy danych, wizualizacji, przetwarzania obrazów i uczenia maszynowego.
- ▶ Ten sam język może służyć do przygotowania danych, wykonania obliczeń, utworzenia wykresów oraz automatyzacji całego procesu analizy.
- ▶ Python współpracuje z kodem napisanym m.in. w C, C++ i Fortranie, dlatego często pełni rolę warstwy łączącej wydajne biblioteki numeryczne.
- ▶ Jest wieloplatformowy i rozwijany jako oprogramowanie otwarte.

Zastosowania Pythona w fizyce medycznej

- ▶ analiza danych pomiarowych i statystyczna ocena wyników,
- ▶ przetwarzanie danych medycznych oraz automatyzacja powtarzalnych zadań,
- ▶ odczyt i analiza obrazów medycznych, m.in. w formatach DICOM i NIfTI,
- ▶ modelowanie matematyczne, symulacje numeryczne i optymalizacja,
- ▶ analiza rozkładów dawki i danych związanych z radioterapią,
- ▶ segmentacja obrazów, klasyfikacja i wykrywanie anomalii,
- ▶ tworzenie powtarzalnych analiz i dokumentowanie wyników badań.

Sekcja 4

Uruchamianie Pythona

REPL

REPL: Akronim od **Read-Eval-Print Loop**; interaktywne środowisko programistyczne umożliwiające wprowadzanie kodu, jego natychmiastowe wykonanie i wyświetlanie wyników.

Python jest językiem interpretowanym, co sprawia, że REPL jest integralną częścią jego ekosystemu, ułatwiając szybkie prototypowanie i testowanie kodu.

Działanie REPL:

- ▶ **Read:** Odczyt wprowadzonego kodu przez użytkownika.
- ▶ **Eval:** Ewaluacja (wykonanie) kodu przez interpreter.
- ▶ **Print:** Wyświetlenie wyników ewaluacji.
- ▶ **Loop:** Powtarzanie procesu dla kolejnych komend.

Zalety korzystania z REPL:

- ▶ Natychmiastowa informacja zwrotna.
- ▶ Ułatwione debugowanie i eksploracja bibliotek.
- ▶ Wspomaganie nauki języka i testowania hipotez programistycznych.

Prosty Edytor i Terminal

Pisanie kodu:

- ▶ Użyj dowolnego prostego edytora tekstu, takiego jak Notatnik (Windows), TextEdit (macOS) lub Gedit (Linux).
- ▶ Zapisz plik z rozszerzeniem .py, np. skrypt.py.

Uruchamianie skryptu:

- ▶ Otwórz terminal lub wiersz poleceń.
- ▶ Przejdź do katalogu zawierającego skrypt za pomocą polecenia `cd`.
- ▶ Wpisz `python skrypt.py` lub `python3 skrypt.py`, aby uruchomić skrypt.

Zalety:

- ▶ Prosta konfiguracja bez dodatkowych narzędzi.
- ▶ Pomaga zrozumieć podstawy działania Pythona i środowiska systemowego.

Wady:

- ▶ Brak zaawansowanych funkcji edytorskich, takich jak autouzupełnianie czy debugowanie.
- ▶ Mniej efektywne dla większych projektów.

Visual Studio Code (VS Code)

Pisanie kodu:

- ▶ VS Code to darmowe, wieloplatformowe środowisko programistyczne.
- ▶ Zainstaluj rozszerzenie Python od Microsoft, które oferuje: podświetlanie składni, autouzupełnianie kodu (IntelliSense), debugger, integrację z Jupyter Notebook.

Uruchamianie skryptu:

- ▶ Wbudowany terminal pozwala na uruchamianie skryptów bezpośrednio z edytora.
- ▶ Debugger umożliwia wykonywanie kodu krok po kroku, ustawianie breakpointów i analizę zmiennych.

Zalety:

- ▶ Bogate funkcje ułatwiające pisanie i debugowanie kodu.
- ▶ Możliwość personalizacji poprzez liczne rozszerzenia.
- ▶ Integracja z systemami kontroli wersji (np. Git).

Wady:

- ▶ Może wymagać początkowej konfiguracji.
- ▶ Wyższe zużycie zasobów systemowych w porównaniu z prostymi edytorami.

Jupyter Notebook

Pisanie kodu:

- ▶ Interaktywne środowisko uruchamiane w przeglądarce internetowej.
- ▶ Dokumenty są podzielone na komórki zawierające kod lub tekst sformatowany w Markdown.

Uruchamianie skryptu:

- ▶ Komórki z kodem mogą być uruchamiane indywidualnie, co umożliwia iteracyjne testowanie i modyfikowanie kodu.
- ▶ Wyniki są wyświetlane bezpośrednio pod komórkami, włączając w to wykresy i dane wyjściowe.

Zalety:

- ▶ Idealne narzędzie do analizy danych, uczenia maszynowego i wizualizacji.
- ▶ Ułatwia dokumentowanie procesu pracy i prezentowanie wyników.
- ▶ Wspiera wiele języków programowania poprzez różne kernelle.

Wady:

- ▶ Nie jest optymalne dla tworzenia aplikacji produkcyjnych.
- ▶ Może prowadzić do nieczytelności kodu przy nieodpowiednim zarządzaniu komórkami.

Sekcja 5

Podstawowe typy danych, operacje wejścia wyjścia

Podstawowe typy danych

Do podstawowych typów danych zaliczamy:

- ▶ Typ `int` (integer) - liczba całkowita (dodatnia lub ujemna) o dowolnej wartości.

```
1 a = -2000
2 b = 314159265358979323846264338327950288419716939937510582
```

- ▶ Typ `float` (zmiennoprzecinkowy) - liczba rzeczywista posiadająca część całkowitą i ułamkową. Separatorem dziesiętnym jest kropka.

```
1 e = 2.718281828459045
2 earth_mass = 5.972e24
3 atomic_mass_constant = 1.661e-27
```

Podstawowe typy danych

- Typ `complex` (zespolony) - liczba zespolona zapisywana jako suma części rzeczywistej i części urojonej. Wartość urojoną oznacza się poprzez listerę *j*. Część rzeczywista i urojona są przechowywane jako wartości typu `float`.

```
1 c1 = 1 + 2j
2 r = c1.real
3 i = c1.imag
4 c2 = complex(0.5, -1.1)
```

- Typ `bool` (logiczny) - reprezentacji wartości logicznej `True` lub `False`.

```
1 p1 = True
2 p2 = False
```

Podstawowe typy danych

- Typ `str` (tekstowy) - ciąg danych znakowych, łańcuch znaków. Może być zawarty w cudzysłowie lub apostrofie, ale notacja musi być konsekwentna, to znaczy musi się zaczynać i kończyć tym samym znakiem (cudzysłowem lub apostrofem).

```
1 s1 = 'Wykład z Pythona.'  
2 s2 = "Wykład z Pythona."
```

- Typ `NoneType` (null) - obiekt reprezentujący brak wartości (odpowiednik `null` z innych języków).

```
1 n = None
```

Do sprawdzenia typu danych obiektu służy polecenie `type()`.

```
1 a = 42  
2 type(a)
```

Operacje wejścia wyjścia

Do pobierania danych wejściowych z klawiatury służy funkcja `input()`. Natomiast do wypisywania tekstu (lub innego obiektu) na ekranie konsoli służy funkcja `print()`.

```
1 name = input('Podaj swoje imię: ')\n2 print('Masz na imię: ', name)
```

Funkcja `print()` domyślnie przechodzi do nowej linii po wydrukowaniu zdanego napisu, aby tego uniknąć należy użyć następującego argumentu:

```
1 print('Wykład z fizyki.',end='')
```

W wypisywanym tekście mogą również występować znaki specjalnie, np.:

- ▶ `\n` - przejście do nowej linii,
- ▶ `\t` - tabulator (odstęp),
- ▶ `\'` oraz `\"` - pojedynczy oraz podwójny cudzysłów.

Operacje wejścia wyjścia

Domyślnym typem wprowadzanych danych w funkcji `input()` są dane typu `str`. Jeśli chcemy aby wprowadzone dane reprezentowały tryb liczbowy należy je przekonwertować.

```
1 c = int(input('Podaj liczbę całkowitą: '))
2 r = float(input('Podaj liczbę rzeczywistą: '))
```

f-string - formatowane literały ciągów znakowych

Od wersji Pythona 3.6 możliwe jest bardzo wygodne interpolowanie łańcuchów znakowych, wystarczy poprzedzić łańcuch literą `f`.

```
1 p = 3.1415926536
2 print(f'Przybliżona wartość liczby pi to {p:.4f}.')
3 #wcześniejszy sposób zapisu
4 print('Przybliżona wartość liczby pi to {:.4f}'.format(p))
5 print('Przybliżona wartość liczby pi to %.4f' % (p))
```

Natomiast formatowanie wyświetlania może być zrealizowane jako:

```
1 print(f'{name:10} ==> {phone:10d}')
```

f-string – przykłady

Składnia: f'Tekst {wyrażenie:format}'. Część :format jest opcjonalna.

```
1 name, n = 'Anna', 7
2 x = 12.3456
3
4 print(f'Witaj, {name}!')      # Witaj, Anna!
5 print(f'n + 1 = {n + 1}')    # n + 1 = 8
6 print(f'{x:.2f}')            # 12.35
7 print(f'{x:.2e}')            # 1.23e+01
8 print(f'{0.875:.1%}')        # 87.5%
9 print(f'{n:04d}')            # 0007
10 print(f'|{name:<8}|')        # |Anna      |
11 print(f'|{n:>8d}|')          # |          7|
12 print(f'{{n}} = {n}')
```

- ▶ W {...} Python oblicza wyrażenie i wstawia jego wartość do tekstu.
- ▶ Po : określamy format: .2f – dwa miejsca po kropce, .2e – zapis naukowy, .1% – procenty.
- ▶ <, >, ^ oznaczają wyrównanie do lewej, prawej i do środka. Szerokość pola jest minimalna.
- ▶ Formatowanie zmienia tekst wyniku, nie wartość zmiennej. {{ i }} dają dosłowne klamry.

Sekcja 6

Operator

Operatory arytmetyczne

Operator	Funkcja
$x + y$	dodawanie
$x - y$	odejmowanie
$x * y$	mnożenie
x / y	dzielenie
$x // y$	dzielenie całkowite
$x ** y$	potęgowanie (x do potęgi y)
$x \% y$	reszta z dzielenia (x modulo y)
$+x$	jednoargumentowe dodawanie
$-x$	jednoargumentowe odejmowanie

- ▶ Operator dzielenia $/$ tworzy liczbę zmiennoprzecinkową, jeśli argumentami są liczby całkowite.
- ▶ Operator dzielenia całkowitego $//$ zaokrągla iloraz w dół. Działa z liczbami całkowitymi i zmiennoprzecinkowymi.
- ▶ Operator modulo $\%$ zwraca resztę z dzielenia. W przypadku liczb zmiennoprzecinkowych operator ten zwraca zmiennoprzecinkową resztę z działania $x - (x // y) * y$.

Operatory porównania i logiczne

Operatory porównania

Operator	Opis
$x == y$	równe
$x != y$	nie jest równe
$x < y$	mniejsze niż
$x > y$	większe niż
$x \leq y$	mniejsze lub równe
$x \geq y$	większe lub równe

- Wynikiem porównania jest wartość logiczna `True` lub `False`.

Operatory logiczne

Operator	Opis
$x \text{ and } y$	koniunkcja
$x \text{ or } y$	alternatywa
$\text{not } x$	negacja

- Operatory logiczne mają swoją kolejność wykonywania najwyższy priorytet ma operator `not`, następnie `and`, a najniższy priorytet ma operator `or`.

Sekcja 7

Wcięcia i instrukcje warunkowe

Wcięcia

Python automatycznie wykrywa granice bloków kodu poprzez wcięcia wierszy - czyli pustą przestrzeń po lewej stronie. Standardowe wcięcie to 4 spacje. Można też używać tabulatora, ale sposób wcięć musi być spójny w ramach programu.

```
1 x = 10
2 if x > 5:
3     for i in range(3):
4         print(f"x jest większe od 5, a i = {i}")
```

Przykład błędów we wcięciach

```
1     x = 'ogród'           # Błąd - wcięcie pierwszego wiersza
2 if 'płot' in 'żywopłot':
3     print(x * 8)
4     x += 'NI'             # Błąd - nieoczekiwane wcięcie
5     if x.endswith('NI'):
6         x *= 2
7     print(x)              # Błąd - niespójne wcięcia
```

► Jeżeli poziomy wcięć nie będą spójne Python zgłosi błąd `IndentationError`.

Instrukcje warunkowe

- ▶ Instrukcja warunkowa `if` pozwala na wykonywanie fragmentów kodu tylko wtedy, gdy spełniony jest określony warunek logiczny.
- ▶ W Pythonie warunek może być dowolnym wyrażeniem, które zostanie ocenione jako typ `bool` (prawda/fałsz).

```
1 if warunek: #Wyrażenie logiczne, które zostanie ocenione jako True lub False.  
2     blok_kodu #Fragment kodu wykonany tylko wtedy, gdy warunek jest spełniony (True).
```

```
1 x = 5  
2 if x > 3:  
3     print("x jest większe od 3")
```

- ▶ Instrukcja `else` umożliwia wykonanie alternatywnego bloku kodu, jeśli warunek w `if` jest fałszywy.

```
1 if warunek:  
2     blok_kodu_1  
3 else:  
4     blok_kodu_2
```

Instrukcje warunkowe

- Instrukcja `elif` (`else if`) pozwala na sprawdzenie kolejnych warunków, jeśli pierwszy warunek `if` nie jest spełniony.

```
1 if warunek1:
2     blok_kodu_1
3 elif warunek2:
4     blok_kodu_2
5 else:
6     blok_kodu_3
```

Przykład:

```
1 x = 7
2 if x > 10:
3     print("x jest większe od 10")
4 elif x > 5:
5     print("x jest większe od 5, ale mniejsze lub równe 10")
6 else:
7     print("x jest mniejsze lub równe 5")
```

- Można mieć dowolną liczbę instrukcji `elif`, ale tylko jeden blok `else` (który jest opcjonalny).

Sekcja 8

Przykłady

Przykłady

```
1 number1 = input('Podaj pierwszą liczbę: ')
2 number2 = input('Podaj drugą liczbę: ')
3 result = number1 * number2
4 print(f'Wynik: {result}')
```

```
1 number1 = input('Podaj pierwszą liczbę: ')\n2 number2 = input('Podaj drugą liczbę: ')\n3 result = number1 * number2\n4 print(f'Wynik: {result}')
```

TypeError: can't multiply sequence by non-int of type 'str'

```
1 number1 = input('Podaj pierwszą liczbę: ')
2 number2 = input('Podaj drugą liczbę: ')
3 result = number1 * number2
4 print(f'Wynik: {result}')
```

TypeError: can't multiply sequence by non-int of type 'str'

```
1 number1 = int(input('Podaj pierwszą liczbę: '))
2 number2 = int(input('Podaj drugą liczbę: '))
3 result = number1 * number2
4 print(f'Wynik: {result}')
```

```
1 number1 = input('Podaj pierwszą liczbę: ')
2 number2 = input('Podaj drugą liczbę: ')
3 result = number1 * number2
4 print(f'Wynik: {result}')
```

TypeError: can't multiply sequence by non-int of type 'str'

```
1 number1 = int(input('Podaj pierwszą liczbę: '))
2 number2 = int(input('Podaj drugą liczbę: '))
3 result = number1 * number2
4 print(f'Wynik: {result}')
```

```
1 number1 = float(input('Podaj pierwszą liczbę: '))
2 number2 = float(input('Podaj drugą liczbę: '))
3 wynik = number1 * number2
4 print(f'Wynik: {result}')
```

Przykłady

```
1 test = False
2 if test:
3     print("Prawda")
4 else:
5     print("Fałsz")
```

Przykłady

```
1 test = False
2 if test:
3     print("Prawda")
4 else:
5     print("Fałsz")
```

Fałsz

Przykłady

```
1 test = False
2 if test:
3     print("Prawda")
4 else:
5     print("Fałsz")
```

Fałsz

```
1 test = 4
2 if test:
3     print("Prawda")
4 else:
5     print("Fałsz")
```

Przykłady

```
1 test = False
2 if test:
3     print("Prawda")
4 else:
5     print("Fałsz")
```

Fałsz

```
1 test = 4
2 if test:
3     print("Prawda")
4 else:
5     print("Fałsz")
```

Prawda

Przykłady

```
1 test = False
2 if test == True:
3     print("Prawda")
4 else:
5     print("Fałsz")
```

Przykłady

```
1 test = False
2 if test == True:
3     print("Prawda")
4 else:
5     print("Fałsz")
```

Fałsz

Przykłady

```
1 test = False
2 if test == True:
3     print("Prawda")
4 else:
5     print("Fałsz")
```

Fałsz

```
1 test = 4
2 if test == True:
3     print("Prawda")
4 elif not test:
5     print("Fałsz")
6 else:
7     print("Inna wartość")
```

Przykłady

```
1 test = False
2 if test == True:
3     print("Prawda")
4 else:
5     print("Fałsz")
```

Fałsz

```
1 test = 4
2 if test == True:
3     print("Prawda")
4 elif not test:
5     print("Fałsz")
6 else:
7     print("Inna wartość")
```

Inna wartość

Przykłady

```
1 word = input('Podaj słowo: ')
2 letter = input('Podaj literę: ')
3
4 if letter in word:
5     print(f"Litera '{letter}' znajduje się w słowie '{word}'")
6 else:
7     print(f"Litera '{letter}' nie znajduje się w słowie '{word}'")
```

```
1 output = False or False or True
2 print(output)
```

True

```
1 output = False and False and True
2 print(output)
```

False

Sekcja 9

Pętle

Pętla while

Pętla `while` w Pythonie to jedna z podstawowych konstrukcji służących do realizacji iteracji, czyli powtarzania określonego bloku kodu tak długo, jak spełniony jest dany warunek logiczny.

```
1 while warunek:
2     blok_kodu
```

Pętla `while` sprawdza wyrażenie warunkowe przed każdą iteracją. Jeżeli wyrażenie jest **prawdziwe**, kod wewnątrz pętli zostanie wykonany. Gdy wyrażenie stanie się **fałszywe**, pętla się kończy i program kontynuuje swoje działanie od następnej instrukcji po pętli.

```
1 counter = 0
2 while counter < 5:
3     print(counter)
4     counter += 1
```

Jeśli warunek pętli zawsze pozostaje prawdziwy, pętla nigdy się nie zakończy. Nazywamy to **pętlą nieskończoną**. W Pythonie nieskończoną pętlę można stworzyć w następujący sposób:

```
1 while True:
2     # Kod wykonywany w nieskończoność
```

Instrukcje sterujące w pętli while

break: Instrukcja ta powoduje natychmiastowe przerwanie pętli, niezależnie od warunku.

```
1 while True:
2     user_input = input("Podaj liczbę: ")
3     if user_input == 'stop':
4         break
```

continue: Pomija resztę kodu w bieżącej iteracji i przechodzi do następnej iteracji, bez przerywania pętli.

```
1 counter = 0
2 while counter < 10:
3     counter += 1
4     if counter % 2 == 0:
5         continue
6     print(counter)
```

W tym przykładzie `continue` powoduje, że liczby parzyste nie będą wypisywane, ponieważ dla nich pętla natychmiast przechodzi do kolejnej iteracji, pomijając instrukcję `print`.

Pętla for

Pętla `for` to konstrukcja służąca do iteracji po elementach kolekcji (takich jak listy, krotki, słowniki, zbiory) lub po wynikach generatorów, umożliwiającą wykonanie bloku kodu dla każdego elementu w sekwencji.

```
1 for element in sekwencja:
2     blok_kodu
```

Pętla `for` iteruje po wszystkich elementach w danej sekwencji (kolekcji) lub obiekcie iterowalnym. Dla każdego elementu z sekwencji przypisuje wartość do zmiennej `element`, a następnie wykonuje blok kodu. Proces ten trwa, dopóki wszystkie elementy sekwencji nie zostaną przetworzone.

```
1 numbers = [1, 2, 3, 4, 5]
2 for num in numbers:
3     print(num)
```

Funkcja `range()` - Pętla `for` często jest używana z funkcją `range()`, która generuje ciąg liczb. Składnia `range()` to:

- ▶ `range(stop)` – liczby od 0 do `stop` (bez `stop`).
- ▶ `range(start, stop)` – liczby od `start` do `stop` (bez `stop`).
- ▶ `range(start, stop, step)` – liczby od `start` do `stop` z krokiem `step`.

Pętla for

Przykład z range():

```
1 for i in range(5):  
2     print(i)
```

W pętli for występują te same instrukcje sterujące break i continue.

```
1 for i in range(10):  
2     if i == 5:  
3         break  
4     print(i)  
5 for i in range(5):  
6     if i == 2:  
7         continue  
8     print(i)
```

Pętla for

Pętla for można **zagnieźdźać**, co oznacza, że w ramach jednej pętli for można umieścić inną pętlę for. Jest to szczególnie przydatne przy iteracji po strukturach wielowymiarowych, np. macierzach.

```
1 matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
2 for row in matrix:
3     for num in row:
4         print(num, end=' ')
```

Pętla for może być używana z wieloma typami obiektów jak listy, łańcuchy znaków, zbiory i inne:

```
1 fruits = ['jabłko', 'banan', 'gruszka']
2 for fruit in fruits:
3     print(fruit)
4
5 word = "Python"
6 for char in word:
7     print(char)
8
9 unique_numbers = {1, 2, 3, 4, 5}
10 for number in unique_numbers:
11     print(number)
```

Pętla for

Czasem przydatne jest jednocześnie iterowanie po indeksach i wartościach listy. Można to osiągnąć za pomocą funkcji `enumerate()`:

```
1 fruits = ['jabłko', 'banan', 'gruszka']
2 for index, fruit in enumerate(fruits):
3     print(index, fruit)
```

Zastosowania pętli:

- ▶ Powtarzanie określonych operacji do momentu spełnienia warunku (np. pobieranie danych od użytkownika, aż zostaną wprowadzone poprawne wartości).
- ▶ Przetwarzanie strumieni danych, gdy dane są odbierane w sposób ciągły.
- ▶ Realizacja algorytmów, w których liczba iteracji nie jest z góry znana (np. metody numeryczne).

Sekcja 10

Przykłady

Przykład 1 i 2

```
1 i = 1
2 while i <= 10:
3     print(i)
4 print("Koniec")
```

```
1 i = 1
2 while i <= 10:
3     print(i)
4     i += 1
5 print("Koniec")
```

Przykład 1 i 2

```
1 i = 1
2 while i <= 10:
3     print(i)
4 print("Koniec")
```

```
1 i = 1
2 while i <= 10:
3     print(i)
4     i += 1
5 print("Koniec")
```

```
1 sequence = input('podaj sekwencję')
2 length = len(sequence)
3 i = 0
4 while i < length:
5     print(sequence[i])
6     i += 1
```

```
1 sequence = input('Podaj sekwencję: ')
2 for letter in sequence:
3     print(letter)
```

Przykład 3

```
1 i = 1
2 while i <= 19:
3     print(f"{i*' *': ^19}")
4     i += 2
```

```
      *
    ***
  *****
 *****
*****
*****
*****
*****
```

Przykład 3

```
1 i = 1
2 while i <= 19:
3     print(f"{i}*'*': ~19}")
4     i += 2
```

```
1 i = 1
2 print(f"{i*'*': ^21}")
3 while i <= 10:
4     print(f"{i*'*': >10}|{i*'*': <10}")
5     i += 1
```

```

      *
    ***
  *****
*****
*****
*****
*****
*****

```

```

      *
    * | *
  ** | **
 *** | ***
**** | ****
*****|*****
  
```

Sekcja 11

Zadania dodatkowe

Zadanie dodatkowe

Zadanie (2 punkty)

Proszę napisać prostą grę tekstową z wykorzystaniem instrukcji `if`, `elif` oraz `else`.

Gra powinna zawierać przynajmniej pięć następujących po sobie opcji wyboru. Po każdej akcji program powinien odczytywać decyzję użytkownika. A następnie podejmować dalsze działania zgodnie z przygotowaną historią. Temat gry może być całkowicie dowolny.

Kod programu proszę przesłać na adres: jacek.gatlik@agh.edu.pl